

```
In [2]: #环境设置

import os
import re
import torch
import warnings

os.environ["KMP_DUPLICATE_LIB_OK"] = "True"
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "max_split_size_mb:128"
torch.backends.cudnn.benchmark=True

warnings.filterwarnings("ignore", category=DeprecationWarning)
warnings.filterwarnings("ignore", category=UserWarning)
warnings.filterwarnings("ignore", category=FutureWarning)


import cv2
from skimage import io
from PIL import Image
import torchvision
from torch import nn
from torch import optim
import torchvision.utils as vutils
from torchvision import transforms
from torchvision import models as M
from torchvision import datasets as dest
from torch.utils.data import Dataset, DataLoader
import torch.nn.functional as F
from torchinfo import summary


import matplotlib as mlp
import matplotlib.pyplot as plt
import seaborn as sns
import random
import numpy as np
import pandas as pd
import datetime
from time import time
import gc


from sklearn.model_selection import train_test_split

manualSeed=1412
torch.manual_seed(manualSeed)
```

```
random.seed(manualSeed)
np.random.seed(manualSeed)
```

```
In [3]: torch.cuda.is_available()
```

```
Out[3]: True
```

```
In [4]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
In [5]: device
```

```
Out[5]: device(type='cuda')
```

```
In [6]: !nvidia-smi -L
```

```
GPU 0: NVIDIA GeForce RTX 2070 SUPER (UUID: GPU-ddd6b802-f52d-1fc7-5bf2-dc2c56c5f
d41)
```

```
In [7]: torch.cuda.memory_reserved()
```

```
Out[7]: 0
```

```
In [8]: torch.cuda.memory_allocated()
```

```
Out[8]: 0
```

1 图像分割任务的必备基础

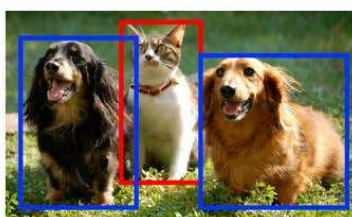
图像分割 (image Segmentation) 是深度学习在图像领域的重要应用之一，如果说识别任务是针对一张图像进行的学习、检测任务是针对图像中不同的对象进行的学习，那分割任务就是针对单一像素进行的学习。分割任务是像素级别的有监督任务，在图像分割时，我们需要对图像中的每一个像素进行分类，因此我们可以找出图像中每个对象的“精确边界”。以下面的图像为例，我们可以找出["猫","狗"]两个标签类别所对应的具体对象，并且找出这些对象的精确边界。当原始图像越复杂，分割任务中需要输出的标签类别也就越多，对下面的图像，除了也可以["猫","狗"]分割之外，还可以对["草坪","项圈"]等标签进行分割。

图像识别



CAT

对象/物体检测



CAT, DOG

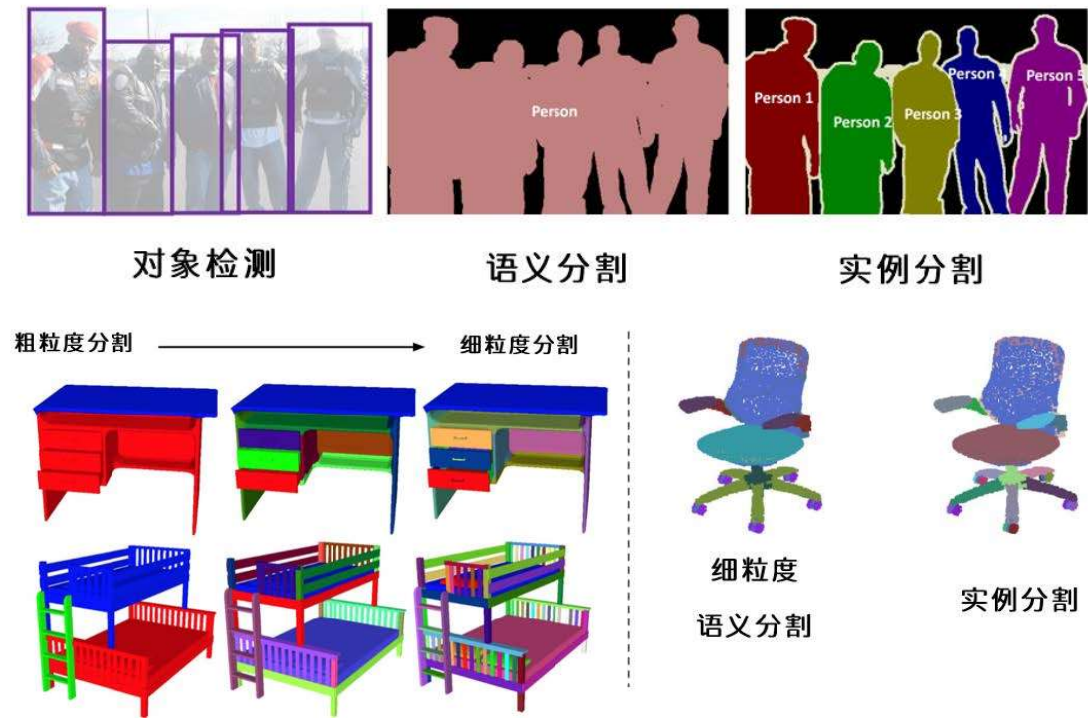
分割



CAT, DOG

从分割的类别来看，我们可以执行将不同性质的物体分开的语义分割 (semantic segmentation)，也可以执行将每个对象都分割开来的实例分割 (instance

segmentation)，还可以执行使用多边形或颜色进行分割的分割方法。同时，根据分割的“细致程度”，还可以分为粗粒度分割（Coarse Segmentation）与细粒度分割（Fine Segmentation），只要拥有对应的标签，我们就可以将图像分割到非常非常精细的程度：



• 分割任务中的标签

不难发现，分割任务具体可以做到的分割程度是由训练图像中的标签决定的，而分割图像中的标签具体是什么样呢？在分割任务中，训练数据是原始图像，原始图像中存在的所有对象都可以被标记为某一类别，而一张图像所对应的标签**一般**是与原始图像相同尺寸的标签矩阵，该矩阵被上色之后被称为“遮罩矩阵”(mask)。通常来说，遮罩矩阵中的颜色数量等同于这张标签上的标签类别数量。



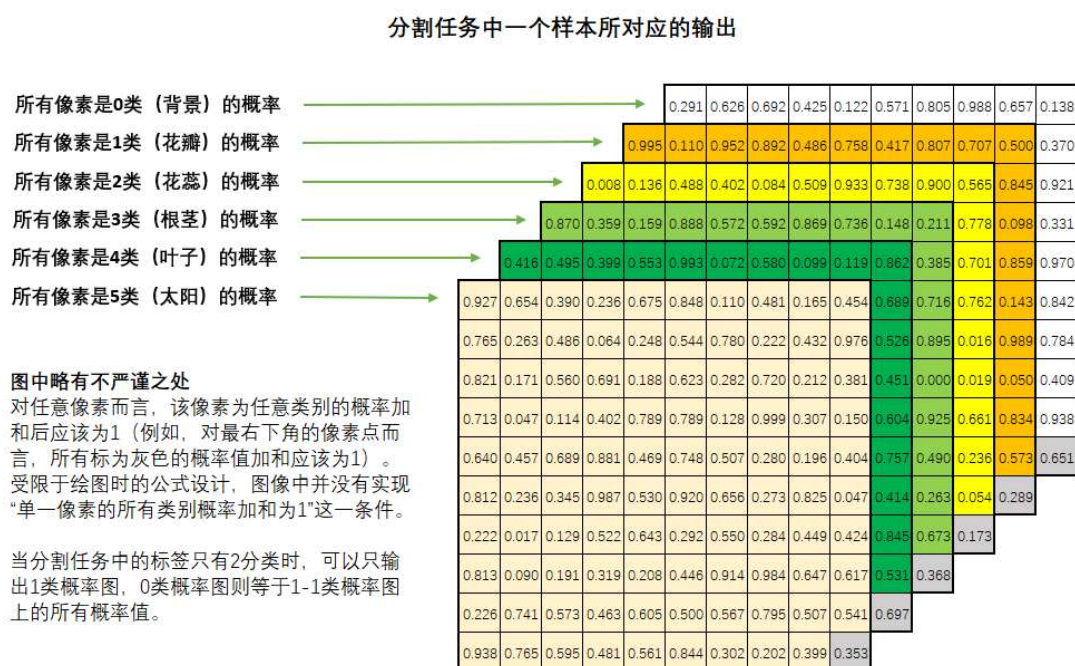
以下面最为简单的10x10尺寸图像为例，假设一张原始图像（一个样本）的结构为(1,10,10)，那它所对应的标签的结构**一般**也为(1,10,10)，假设总共有n_samples个样本，则数据集的标签格式为(n_samples,1,10,10)。



分割架构的输出

当输入标签为图像/矩阵时，你能推断出分割任务中网络对应的输出是什么吗？对于任意用于图像识别的数据集，如果数据有num_classes个标签类别，则神经网络会对每个样本输出num_classes个对应的概率，此时作为输出层的线性层则会有num_classes个神经元，作为输出层的卷积层则会输出num_classes个1x1的特征图。

相对的，在分割任务中，如果数据包含num_classes个标签类别，神经网络则需要对每个像素值输出num_classes个对应的概率。因此，分割任务中一个样本所对应的输出是该样本上所有像素值、在所有标签类别下的概率。具体来看，以10x10的花朵图为例，该数据集的标签有6大类别，因此一个样本所对应的输出就有6张概率图，一张概率图对应着一个类别，反馈着样本上所有的像素是0类、1类、2类.....num_classes类的概率。当输出概率图后，一个像素在所有概率图上最大的概率所对应的类别，就是该像素的预测类别。



需要注意的是，以上图像中有不严谨之处。对一个像素而言，该像素为任意类别的概率加和之后应该为1（例如，对图像上最右下角的像素点而言，所有标位灰色的概率值加和之后应该为1）。在绘制图像时，受限于绘图公式设计，图像中并没有实现“单一像素的所有类别概率加和必须为1”这一条件，但在实际使用数据、输出结果是，这一条件是一定会被满足的。

- 分割任务中的损失函数

同时，还有几个值得注意的问题：

1. **一张图上的存在的标签类别可能少于整个数据集中的标签类别。** 标签类别数量会覆盖整个数据集上的标签类别，因此一张图像上不一定包含了所有的标签类别，但每个样本都必须输出所有类别的概率图。假设一张图像上没有类别A，那我们会期待这张图像所对应的类别A的概率图上的值会全部为0，即没有任何像素属于该类别。
2. **由于需要输出概率图，所以分割网络的输出层一般都是卷积层。** 以此为基础还诞生了整个网络中只有卷积或卷积相关计算的神经网络全卷积网络FCN（fully convolutional network）。
3. **虽然标签和输出都转化为了图像，但我们常用的交叉熵损失等损失函数依然可以使用，**只不过此时我们公式中的预测值是多个矩阵，真实标签也是多个矩阵。具体来看：

二分类交叉熵损失——

$$L = -(y \log p(x) + (1 - y) \log(1 - p(x)))$$

此时 y 就是标签矩阵，而 $p(x)$ 就是对应的概率图，由于都是相同尺寸的图像，所以在进行计算时并无问题。

多分类交叉熵损失，总共有K个类别——

$$L = - \sum_{k=1}^K y_k^* \log(P^k(x))$$

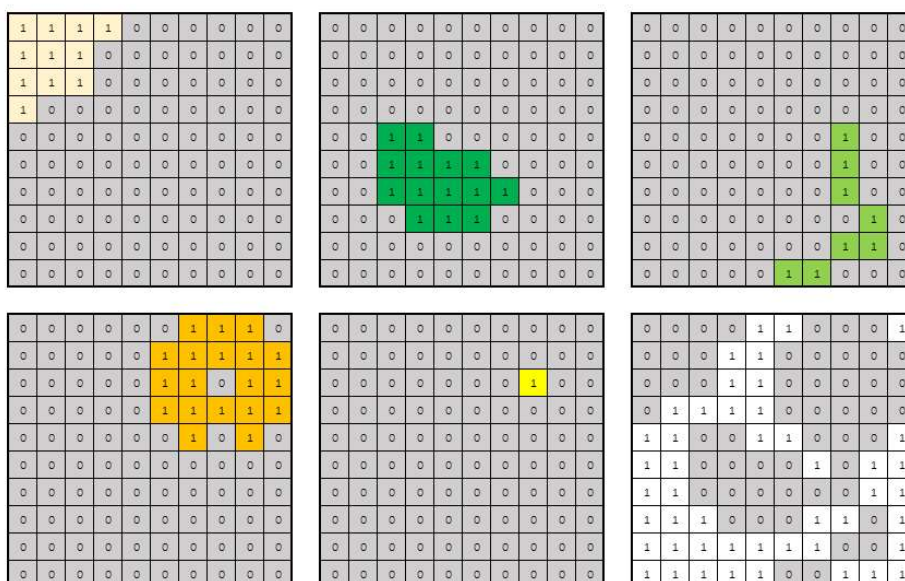
在标签为序列的识别任务中， $P^k(x)$ 是一个样本的类别为 k 的概率，并且 $p^k(x) = \text{Softmax}(\text{网络输出})$ ， y^* 是由真实标签做独热编码后的向量。例如，在3分类情况下，真实标签 y_i 为2时， y^* 为 $[y_1^*, y_2^*, y_3^*]$ ，取值分别为：

$$[y_1^* | y_2^* | y_3^*] = [0 | 1 | 0]$$

而在标签为概率图的分割任务中， $P^k(x)$ 是所有像素的类别为 k 的概率（即概率图），并且 y^* 是将标签矩阵转化为独热形态后的独热矩阵。以之前的花朵图像为例，在 num_classes 个标签分类的情况下，我们会分割出 num_classes 个独热矩阵，如果一个像素的真实标签为该类别，则该像素在

这张独热矩阵上的标签为1，否则标签为0。

一个样本的标签矩阵转化为独热状态后生成的num_classes个矩阵



4. 标签与输出概率图的尺寸必须一致，但有时候标签和输出概率图的尺寸可以略小于原始图像。

图像分割是对每个样本上的每个像素进行分类，因此最为严谨的情况下每个像素都应该有对应的标签，但对像素级别进行分割的根本目的是为了描绘出物体的轮廓。然而，当原始图像的尺寸很大、像素很高时，我们并不需要非常高清的图像才能够展示出物体的轮廓；同时，在许多时候我们需要的只是“大致轮廓”，而不需要非常精确的边缘，因此标签图像可以略小于原始图像。这样的标签可以反馈出每一类对象在原始图像中的大致轮廓，但不能精确地反馈出原始图像的每一个像素点的类别。



标签需要被放置到损失函数中使用，为了能够和标签图像进行像素值——对应的计算，分割架构输出的概率图尺寸必须与标签尺寸一致。因此当标签小于原始图像时，概率图也会小于原始图像。并且，标签和概率图越小，其投射到原始图像上的轮廓就越不准确，依据我们的使用场景，我们可以调整网络架构的输出：

较为粗糙的分割
(概率图/标签可以大幅度小于原始图像)



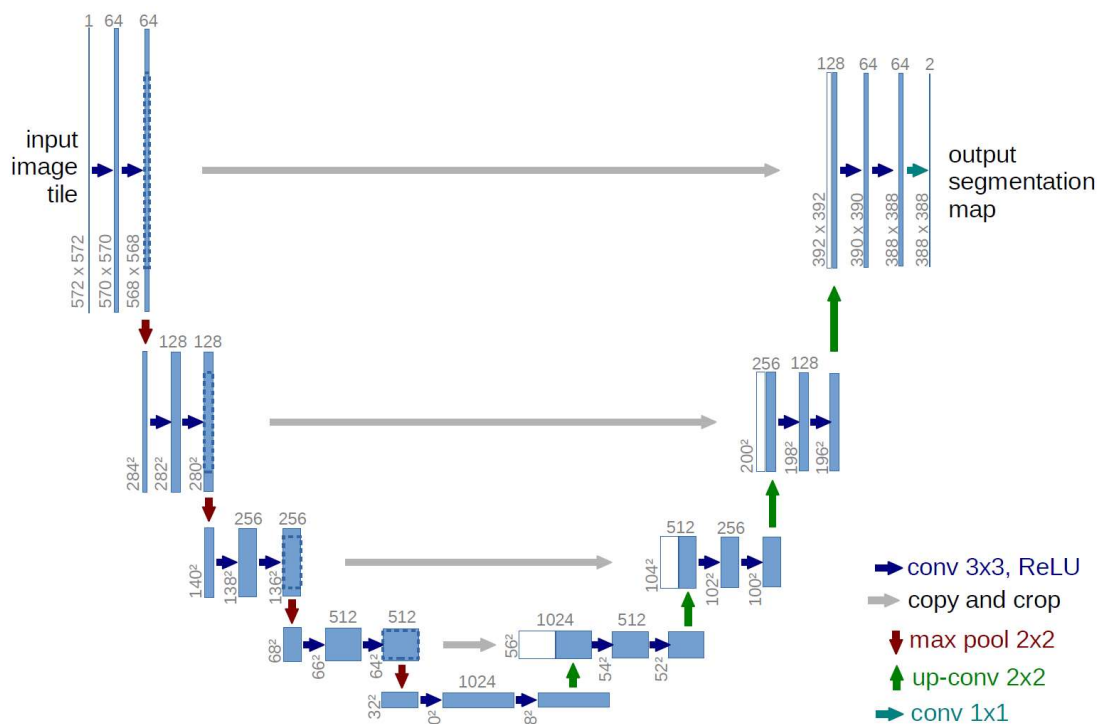
更加精准地分割
(概率图/标签等于原始图像, 或略小于原始图像)



现在你已经了解了关于图像分割架构的一些基本特点了。有了这些基础知识，我们在解析各种分割架构的时候就不容易进入迷雾之中，下一节我们将展开聊聊经典分割架构Unet的结构。

2 经典架构Unet

Unet是一个博采众长的架构，它于2015年被德国弗莱堡大学的研究团队提出，并在原始论文当中被用于生物医学影像的分割。但事实上，它既可以完成分类任务，也可以完成分割任务，还可以被当做无监督算法使用，这是因为它汲取了大量其他网络的精华结构、并且有机地将这些架构融合在了一起。具体地来看，Unet的架构图如下。我们可以从图例、架构上的数字取得不少关键信息。



整个架构图呈现对称的“U字型”，因此该网络被称为Unet。查看架构细节，不难发现Unet其实拥有多重身份和多重标签：

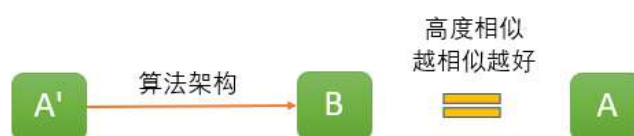
1. **Unet是一个全卷积网络 (FCN)**，它没有使用除了卷积、池化和转置卷积之外的任何层，架构中的数据始终都是四维的图像。
2. **Unet是一个Encoder-Decoder**，它的结构与深度卷积自动编码器高度相似：输入大图像、使用卷积层和池化层组成的Encoder将图像压缩，形成数据隐式表示Latent Representation，接着又使用由卷积层和转置卷积层组成的Decoder将图像放大，最终输出大图像。这一结构左右对称、两边大中间小的结构是自动编码器的典型结构。

灵魂拷问：Encoder-Decoder（也就是Autoencoder，自动编码器）是无监督算法，Unet是有监督算法，难道因为结构一致就可以说Unet是一个自动编码器吗？

事实上，**自动编码器算法不是有监督，却胜似有监督**。以降噪自动编码器为例，首先我们具有一组原始数据A，为了加大自动编码器的训练难度、防止自动编码器直接把输入信息原封不动搬到输出层，我们需要在原始数据A基础上加上噪音，构成带噪音的数据A'。

在训练过程中，我们对编码器输入A'，让架构从A'生成数据B，并且我们对架构的要求是：数据B要尽量与无噪音的数据A相似，这样架构就拥有了“降噪”的能力。这一过程看似平常，但其实已经**等同于一个有监督的过程**了：

无监督的降噪自动编码器：



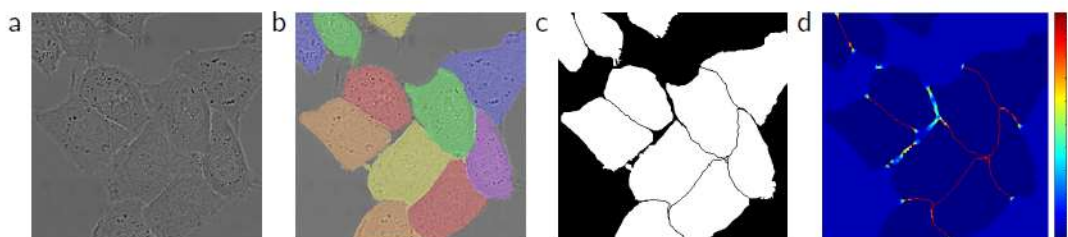
任意有监督算法：



不难发现，自动编码器中损失函数衡量A与B之间的差异，而有监督算法当中损失函数衡量预测标签yhat与真实标签y之间的差异，无论我们面对的任务是给图像上色、还是给图像补全、还是其他任务，只要认为A是真实标签、A'是特征矩阵，那自动编码器就可以被当成一个有监督算法使用。如果想要为黑白图像上色，你应该准备彩色图像作为原始数据A，黑白图像作为输入数据A'，如果你想要将冬天的照片变成夏天，那你应该准备夏天的照片作为原始数据A，冬天的照片作为原始数据A'。至此，专用于“数据表示”的无监督算法就成为了可以“依葫芦画瓢”的有监督算法了，真是妙哉。因此，认为Unet是一个自动编码器没有任何问题。

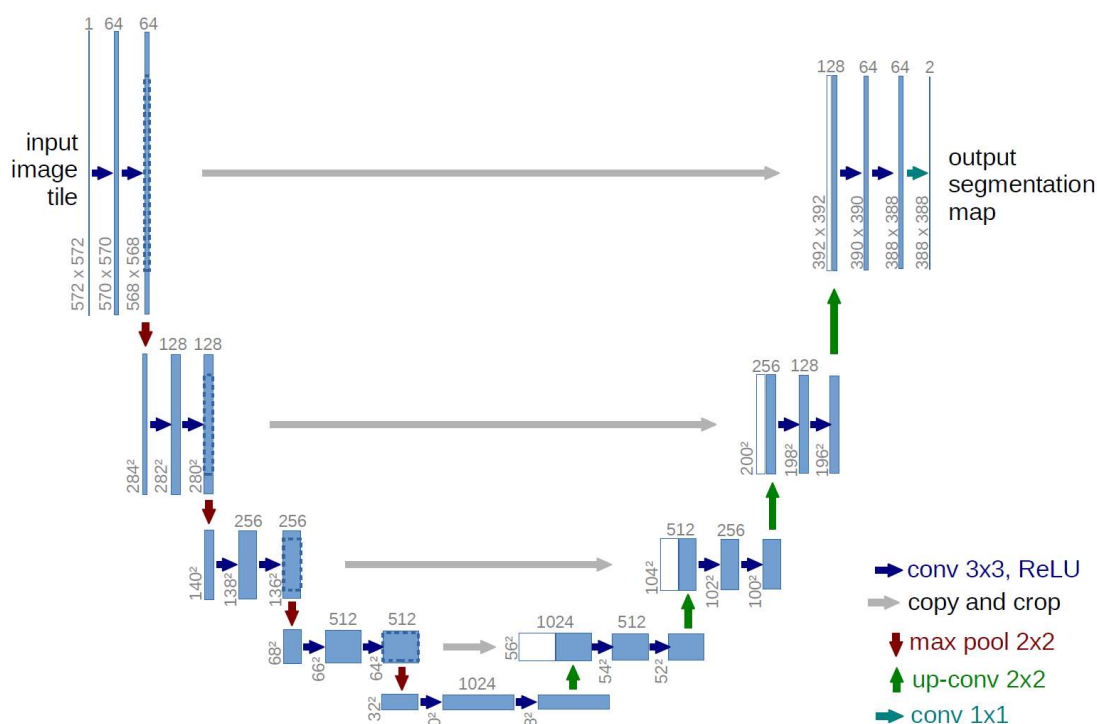
3. **Unet是一个分割网络**，因此它所使用的标签是遮罩，要输出的是概率图，使用的损失函数是二分类交叉熵损失。从最终输出的结果来看，Unet被创造的原始论文中所使用

的分割图像应该是二分类的分割图像。同时，网络输入图像尺寸与输出图像尺寸差异较大，因此可以判断原始论文中所使用的生物医学影像并不需要太高的轮廓精度，检查原始论文，会进一步印证我们的观点：



4. **Unet使用了跳跃链接**，直接将Encoder中尚未压缩完毕的图像传向Decoder，这样可以将更多原始信息直接传向输出方向，帮助生成与原始数据更相似的图像。经过跳跃链接传到Decoder的图像需要与经过编码后的数据合并，再进行卷积，这样Decoder在输出最终的概率图时可以参考的信息就变得更加丰富了。甚至，我们说Unet就是使用了跳跃链接的、有监督版本的自动编码器。

接下来，让我们来复现一下Unet架构。



在这三天的课程当中，我们不断强调了一个事实：在深度学习的后期，许多时候架构中的具体层已经不是最为关键的内容了，相对的，**数据如何在架构中流通才是我们真正关心的内容**。观察架构图，我们可以将Unet总结为如下结构：

输入 → 【双卷积 + 池化】x4 → 【双卷积】 → 【转置卷积 + 双卷积】x4 → 【1x1卷积】
→ 输出

[-----Encoder-----] [bottleneck] [-----Decoder-----] [Output]

因此毫无疑问的，我们可以先定义一个双卷积的结构。在原始论文中规定，无论是在Encoder还是Decoder当中，每个卷积层后面都跟ReLU激活函数。同时，作为卷积架构的惯例，我们在每个卷积层后面、ReLU激活函数之前使用Batch Normalization。需要注意的是，Unet架构中使用的不是令特征图尺寸保持不变的卷积层，而是每经过一个卷积就会将特征图长宽缩小2的卷积层。

2 Network Architecture

The network architecture is illustrated in Figure 1. It consists of a contracting path (left side) and an expansive path (right side). The contracting path follows the typical architecture of a convolutional network. It consists of the repeated application of two 3x3 convolutions (unpadded convolutions), each followed by a rectified linear unit (ReLU) and a 2x2 max pooling operation with stride 2 for downsampling. At each downsampling step we double the number of feature channels. Every step in the expansive path consists of an upsampling of the feature map followed by a 2x2 convolution (“up-convolution”) that halves the number of feature channels, a concatenation with the correspondingly cropped feature map from the contracting path, and two 3x3 convolutions, each followed by a ReLU. The cropping is necessary due to the loss of border pixels in every convolution. At the final layer a 1x1 convolution is used to map each 64-component feature vector to the desired number of classes. In total the network has 23 convolutional layers.

```
In [9]: class DoubleConv2d(nn.Module):
        def __init__(self, in_channels, out_channels):
            super().__init__()
            self.conv = nn.Sequential(nn.Conv2d(in_channels,
                                                out_channels, 3, 1, 0, bias=False)
                                     , nn.BatchNorm2d(out_channels)
                                     , nn.ReLU(inplace=True)
                                     , nn.Conv2d(out_channels,
                                                out_channels, 3, 1, 0, bias=False)
                                     , nn.BatchNorm2d(out_channels)
                                     , nn.ReLU(inplace=True)
                                     )
        def forward(self, x):
            return self.conv(x)
```

接下来我们来考虑如何实现该架构中的数据流。Unet中的数据并不依照从左到右进行线性流动：**因为有跳跃链接的存在，Encoder中的每个双卷积的输出结果都必须被直接传输到Decoder中每个双卷积的输入层。**因此每经过一次双卷积结构我们就需要保存中间结果，因此我们可以按如下方式梳理数据流：

```
In [ ]: #Encoder
        #输入x
```

```

#x = 双卷积(x)
#保存x
#x = 池化层(x)

#x = 双卷积(x)
#保存x
#x = 池化层(x)

#x = 双卷积(x)
#保存x
#x = 池化层(x)

#x = 双卷积(x)
#保存x
#x = 池化层(x)

```

In []: #完全等同于

```

l = []

for i in range(4):
    #x = 双卷积(x)
    #l.append(x)
    #x = 池化层(x)

```

#在这个循环中，4个卷积层的输入特征图数量和输出特征图数量不一致，因此实际上是4个不同的双卷积结构

#但相对的，池化层是完全一致的Maxpool(2)，且池化层没有需要迭代的权重，因此可以被重复使用

#所以，我们可以将4个卷积层定义成一个序列，并单独定义池化层

```

for i in range(4):
    #x = 双卷积[i](x)
    #l.append(x)
    #x = 池化层(x)

```

```

#l = [x1, x2, x3, x4]

```

相似地，**由于跳跃链接的存在，我们需要将数据与跳跃链接传过来的数据合并后，才能输入到Decoder中的每个双卷积层。**并且需要注意的是，在Encoder中池化层是位于双卷积的后面，但在Decoder中转置卷积层是位于双卷积层的前面：

输入 → 【双卷积 + 池化】x4 → 【双卷积】 → 【转置卷积 + 双卷积】x4 → 【1x1卷积】
→ 输出

[-----Encoder-----] [bottleneck] [-----Decoder-----] [Output]

```
In [ ]: #Decoder
#l
#x此时是瓶颈结构的输出

#x = 转置卷积(x)
#x = 合并(x, x4)
#x = 双卷积(x)

#x = 转置卷积(x)
#x = 合并(x, x3)
#x = 双卷积(x)

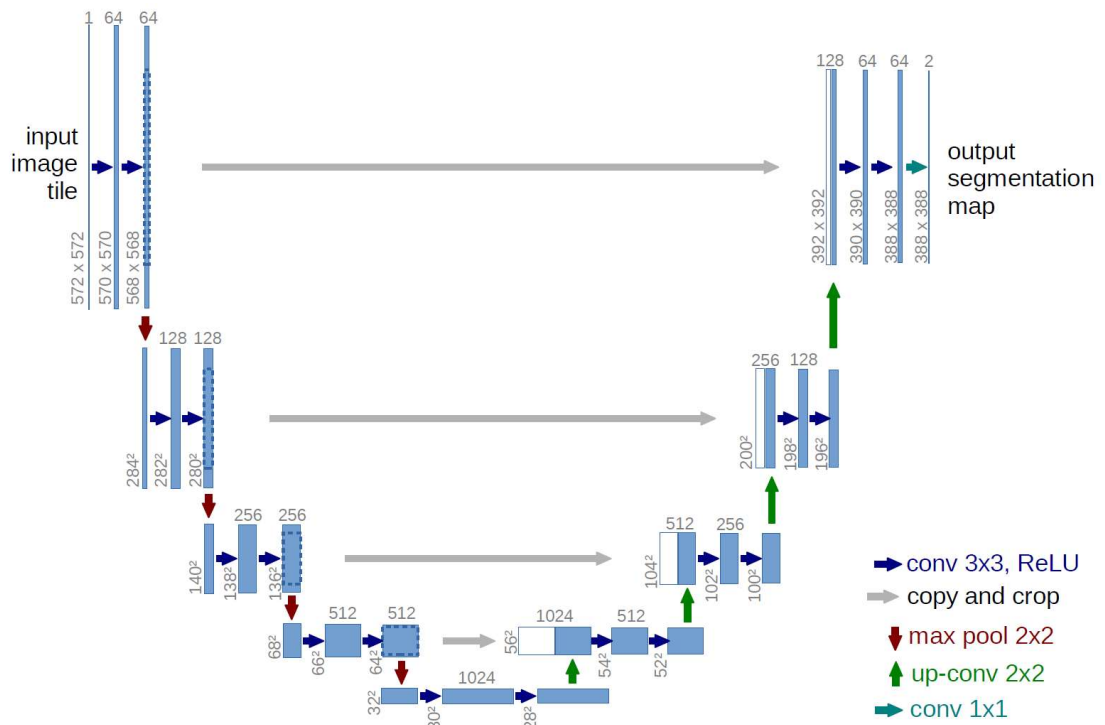
#x = 转置卷积(x)
#x = 合并(x, x2)
#x = 双卷积(x)

#x = 转置卷积(x)
#x = 合并(x, x1)
#x = 双卷积(x)
```

```
In [ ]: #按循环写：

for i in range(4):
    #x = 转置卷积[i](x)
    #x = 合并(x, l[-i])
    #x = 双卷积[i](x)

#其中4个转置卷积的参数不同，4个双卷积的参数也不同，因此都需要构建成序列
```

- 架构复现

```
In [12]: class Unet(nn.Module):
def __init__(self):
super().__init__()

self.encoder_conv = nn.Sequential(DoubleConv2d(1, 64)
, DoubleConv2d(64, 128)
, DoubleConv2d(128, 256)
, DoubleConv2d(256, 512)
)

self.encoder_down = nn.MaxPool2d(2)

self.decoder_up =
nn.Sequential(nn.ConvTranspose2d(1024, 512, 4, 2, 1)
, nn.ConvTranspose2d(512, 256, 4, 2, 1)
, nn.ConvTranspose2d(256, 128, 4, 2, 1)
, nn.ConvTranspose2d(128, 64, 4, 2, 1)
)
```

```

self.decoder_conv = nn.Sequential(DoubleConv2d(1024, 512)
                                   , DoubleConv2d(512, 256)
                                   , DoubleConv2d(256, 128)
                                   , DoubleConv2d(128, 64)
                                   )

self.bottleneck = DoubleConv2d(512, 1024)

self.output = nn.Conv2d(64, 2, 3, 1, 1)

def forward(self, x):

    #encoder: 保存每一个DoubleConv的结果为跳跃链接做准备，同时输出
codes
    skip_connection = []

    for idx in range(4):
        x = self.encoder_conv[idx](x)
        skip_connection.append(x)
        x = self.encoder_down(x)

    x = self.bottleneck(x)

    #调换顺序
    skip_connection = skip_connection[::-1]

    #decoder: codes每经过一个转置卷积，就需要与跳跃链接中的值合并
    #合并后的值进入DoubleConv

    for idx in range(4):
        x = self.decoder_up[idx](x)
        #转换尺寸
        skip_connection[idx] =
transforms.functional.resize(skip_connection[idx], size=x.shape[-2:])
        x = torch.cat((skip_connection[idx], x), dim=1)
        x = self.decoder_conv[idx](x)

    x = self.output(x)
    return x

```

- 架构验证

```
In [11]: net = Unet()  
summary(net, input_size=(10, 1, 572, 572), device="cpu")
```

Out[11]:

Layer (type:depth-idx)	Output Shape	Param #
Unet	--	--
└Sequential: 1	--	--
└DoubleConv2d: 2-1	[10, 64, 568, 568]	--
└Sequential: 3-1	[10, 64, 568, 568]	37,696
└MaxPool2d: 1-1	[10, 64, 284, 284]	--
└Sequential: 1	--	--
└DoubleConv2d: 2-2	[10, 128, 280, 280]	--
└Sequential: 3-2	[10, 128, 280, 280]	221,696
└MaxPool2d: 1-2	[10, 128, 140, 140]	--
└Sequential: 1	--	--
└DoubleConv2d: 2-3	[10, 256, 136, 136]	--
└Sequential: 3-3	[10, 256, 136, 136]	885,760
└MaxPool2d: 1-3	[10, 256, 68, 68]	--
└Sequential: 1	--	--
└DoubleConv2d: 2-4	[10, 512, 64, 64]	--
└Sequential: 3-4	[10, 512, 64, 64]	3,540,992
└MaxPool2d: 1-4	[10, 512, 32, 32]	--
└DoubleConv2d: 1-5	[10, 1024, 28, 28]	--
└Sequential: 2-5	[10, 1024, 28, 28]	--
└Conv2d: 3-5	[10, 1024, 30, 30]	4,718,592
└BatchNorm2d: 3-6	[10, 1024, 30, 30]	2,048
└ReLU: 3-7	[10, 1024, 30, 30]	--
└Conv2d: 3-8	[10, 1024, 28, 28]	9,437,184
└BatchNorm2d: 3-9	[10, 1024, 28, 28]	2,048
└ReLU: 3-10	[10, 1024, 28, 28]	--
└Sequential: 1	--	--
└ConvTranspose2d: 2-6	[10, 512, 56, 56]	8,389,120
└Sequential: 1	--	--
└DoubleConv2d: 2-7	[10, 512, 52, 52]	--
└Sequential: 3-11	[10, 512, 52, 52]	7,079,936
└Sequential: 1	--	--
└ConvTranspose2d: 2-8	[10, 256, 104, 104]	2,097,408
└Sequential: 1	--	--
└DoubleConv2d: 2-9	[10, 256, 100, 100]	--
└Sequential: 3-12	[10, 256, 100, 100]	1,770,496
└Sequential: 1	--	--
└ConvTranspose2d: 2-10	[10, 128, 200, 200]	524,416
└Sequential: 1	--	--
└DoubleConv2d: 2-11	[10, 128, 196, 196]	--
└Sequential: 3-13	[10, 128, 196, 196]	442,880
└Sequential: 1	--	--
└ConvTranspose2d: 2-12	[10, 64, 392, 392]	131,136
└Sequential: 1	--	--
└DoubleConv2d: 2-13	[10, 64, 388, 388]	--
└Sequential: 3-14	[10, 64, 388, 388]	110,848
└Conv2d: 1-6	[10, 2, 388, 388]	1,154
Total params: 39,393,410		
Trainable params: 39,393,410		
Non-trainable params: 0		
Total mult-adds (T): 2.35		
Input size (MB): 13.09		

Forward/backward pass size (MB): 19926.14

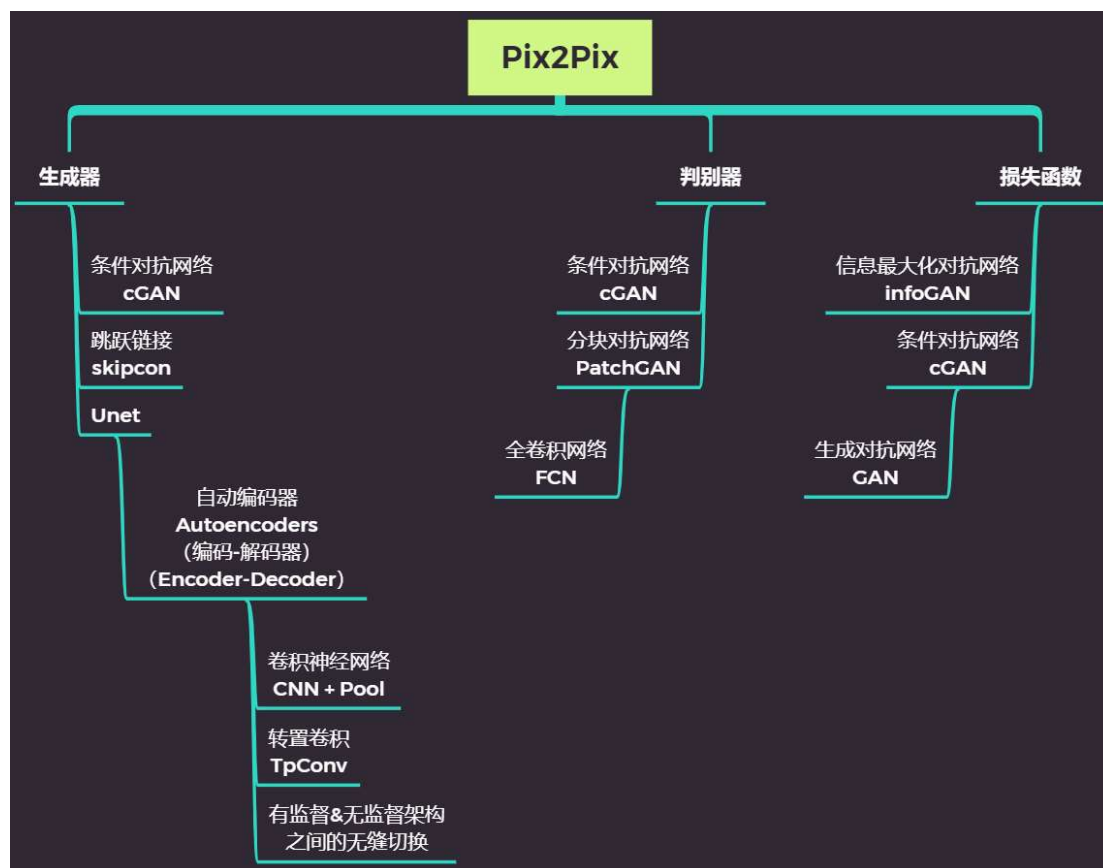
Params size (MB): 157.57

Estimated Total Size (MB): 20096.80

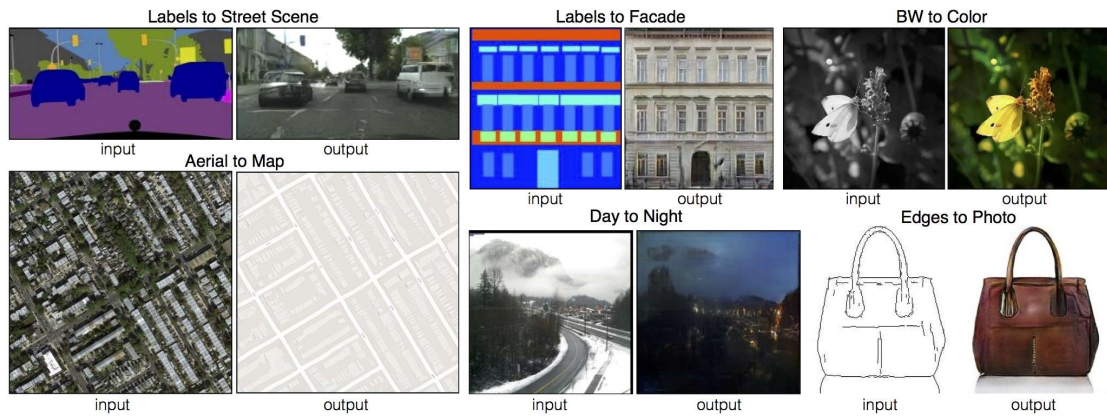
=====

Pix2Pix是论文《[条件对抗网络的图像到图像转换](#)》中提出的经典图像生成架构，它是当代顶级图像模型中最常用、并且最具有代表性的模型之一，这不只是因为Pix2Pix在图像转化方面能力及其强大，更是因为它**集百家之长、融合了深度学习领域众多架构的关键思想，非常适合作为3天生成模型公开课的结尾架构**。从论文的标题来看，很容易就能知道Pix2Pix是一个生成对抗网络，但特别的是，它是一个条件对抗网络，同时它的生成器是Unet，而判别器是基于普通对抗网络改良的PatchGAN、并且它使用了条件生成对抗网络cGAN的损失函数、还使用了信息最大加对抗网络InfoGAN中施加的损失惩罚项。

因此，想要掌握Pix2Pix架构，就需要掌握下面这张思维导图中**全部的架构和思想**，而这些架构和思想的基础正是我们本次3日直播公开课的精髓内容。在正式课程中，我详细的为大家讲解了cGAN和PatchGAN的细节内容，足以帮助大家去复现Pix2Pix架构：



在融合了这么多复杂的架构之后，Pix2Pix在图像生成上的能力及其强大：



• 为什么要使用条件生成对抗网络？

面对复杂的原始数据，普通的GAN甚至DCGAN为什么难以输出完美的结果？这当然与“GAN很难训练”、“GAN对超参数设置高度敏感”，“生成器、判别器在训练中不断动态对抗”等因素有关，但阻碍生成对抗网络输出高质量“假数据”的关键因素之一就是**有效信息的缺失**。在传统生成模型中，GAN与DCGAN的生成器输入都是随机产生的噪音 z ，因此生成器可以使用的唯一信息就是判别器返回的判断结果。该判断结果在理想状态下可以引导生成器不断迭代，但大多数时候并不会告诉生成器“为什么当前数据不够真”或者“应该向什么方向提升”，因此可以想见，判别器给出的反馈信息是相当有限的。当原始数据非常复杂，而判别器又无法传递出更细节的信息时，生成器自然无法生成能够以假乱真的“假数据”。

为了缓解这一问题、并且进一步提升生成对抗网络的模型表现，我们可以为生成器或判别器提供“有效的信息”来辅助模型的对抗表现。这一有效的信息可以是任何与原始数据相关但却不同的信息，例如样本的真实标签、不同模式下的样本数据等等。在相关研究当中，条件生成对抗网络（Conditional Generative Adversarial Nets, cGAN）使用了真实标签作为辅助信息，而信息最大化对抗网络（Information Maximizing Generative Adversarial Nets, infoGAN）则将有效信息推广到了“任意可用”的信息。

• 加入有效信息所带来的改变

我们首先来看cGAN。cGAN可以是使用普通线性层的cGAN，也可以是使用卷积和转置卷积的cCGAN，但无论内部架构使用怎样的网络结构，cGAN的核心思想都是一致的，即将**真实样本realdata的真实标签作为信息输入生成器和判别器，为生成和判别行为做出更好的指导**。更具体地说，**cGAN将真实标签作为特征的一部分，输入生成器和判别器用于训练**。

此时，生成器和判别器就都接收到了来自真实标签的信息，虽然我们难以解释架构具体会如何使用这些信息，但毫无疑问架构是参考了真实标签来完成生成和判别任务的。在这样的情况下，我们可以认为**图像生成和判别是在知晓真实标签的这一条件下完成的**，这就是说，判别器对任意图像所输出的“图像为真”的概率都是基于真实标签的条件概率，生成器生成的任意图像都是知晓该图像真实标签的条件图像。因此，原本的损失函数就由 $V(D,G)$ 转变为了 $V'(D,G)$ ：

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

↓

$$\min_G \max_D V'(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x|y)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z|y)))]$$

虽然损失函数的公式改变了，但GAN架构的输出、损失函数的具体实现方式等都没有发生变化，模型最终输出的依然是一个概率值，只不过在数学上它的性质由普通概率变化为了条件概率。幸运的是，我们不需要在数学上做出修正才能让输出的概率变为条件概率，只要加入条件，这些概率就会自然而然变为条件概率。infoGAN也是使用相似的思路——infoGAN允许任何有效信息被输入到架构中，因此其判别器的输出是参考了“任意有效信息”的条件概率 $D(x|c)$ ，其中 c 表示任意的有效信息。

现在，我们需要考虑的问题有三个：

1. cGAN将标签直接放入架构进行训练，难道不会信息泄露吗？

不会。首先，对无监督网络（生成器）而言标签这一概念并不存在，因此即便把标签信息直接输入无监督网络，网络也不会把这些信息当成是“标准答案”来使用，自然谈不上泄露一说。在生成对抗网络中，由于生成器不具备“常识”，因此在生成器看来，具体标签与生成器需要生成的图像或样本的具体形态并没有**直接关联**：举例说明，因为生成器并不知道数字“7”的写法，因此生成器并不会因为知道标签是“7”就自动生成形如数字7的图像。因此在生成器中不存在信息泄露问题。

而判别器呢？通常来说我们绝不会把标签当做特征输入有监督网络，因为这一定会有信息泄露问题，但在生成对抗网络里面却是一个特例，因为**样本的真实标签与判别器需要判断的标签不是同样的标签**。举例说明，对MNIST数据集而言，图像的标签是0~9的具体数字，而判别器需要判断的却是“图像是否真实”，即便判别器知道了当前图像上的数字是7或者8，也无法直接根据该数字标签得出“图像是真”或“图像是假”的结论。因此，将图像原始的标签放入判别器进行训练，也不会存在信息泄露问题。

这一点也奠定了我们将“其他有效信息”输入架构进行训练的基础。对任意信息来说，只要该信息并不会直接告诉网络“该样本是真”或“该样本是假”，并且又能够一定程度上与真实数据相关，那该信息都可以被输入架构进行训练。

2. 既然生成器和判别器缺乏常识，那它们具体是如何使用这些“有效信息”的？

就像使用其他特征一样地使用。你可能很难想象，但生成器和判别器并不会理解这些信息是“标签”，也并不能理解“有效信息”为什么“有效”，因此它们并不会特别重视或优待这些被输入的“额外的信息”，也因此这些有效信息有时会被生成对抗网络忽略。**对于一个GAN而言，只要能够找出某一标签类别下的共性，并输出满足该共性的图像，就相当于满足了提供的条件了，但这并不能保证生成的图像一定会非常接近真实图像。**

举例说明，如果下面左侧两幅图像和标签被输入到生成对抗网络中，网络可能很快学习到“面包”这一标签下的共性：有复杂纹理，金色黄色棕色为主，根据这些信息生成器最终生成了最右侧的图像。很明显，右侧的图像已经满足了“面包”这一标签下的“棕色”、“纹理”等特点，但图中的内容绝对不是面包。

真实标签：面包



真实标签：面包



生成图像



那如何才能让网络更加重视、更有效地使用被输入地额外信息呢？InfoGAN团队提出了一个解决方案：以某种评估指标衡量生成的图像 $G(x|c)$ 与额外信息 c 之间的相关性，将该相关性定义为 $I(c; G(x|c))$ ，并将 I 作为惩罚项/正则项放入生成对抗网络的损失函数。

具体地来说，当相关性越大， $I(c; G(x|c))$ 越大时，infoGAN的损失函数为：

$$\min_G \max_D V_I(D, G) = V(D, G) - \lambda I(c; G(x|c))$$

相反，当相关性越大， $I(c; G(x|c))$ 越小时，infoGAN的损失函数为：

$$\min_G \max_D V_I(D, G) = V(D, G) + \lambda I(c; G(x|c))$$

如此便可保证生成的假数据 $G(x|c)$ 高度依赖于额外的信息 c ，以此保障 c 能够最大程度被利用，这一点在架构pix2pix中被采用了。

3. 具体如何将标签放入数据进行训练？

标签加入特征进行训练的具体形式与 1)真实数据的结构、 2)标签的结构以及 3)架构可以接受的输入结构有很大的关系，因此标签加入训练的结构是多种多样的。**通常来说，我们会将样本标签与样本特征相匹配后再输入网络**，只有这样才能让标签成为样本的“相关信息”。接下来我们就来看看将标签放入特征进行训练的具体形式。

Pix2Pix架构复现代码如下所示：

- 定义卷积块与转置卷积块

```
In [18]: import torch
import torch.backends.cudnn as cudnn
import torch.nn as nn

def conv_block(in_dim, out_dim):
    return
nn.Sequential(nn.Conv2d(in_dim, in_dim, kernel_size=3, stride=1, padding=1),
              nn.ELU(True),

nn.Conv2d(in_dim, in_dim, kernel_size=3, stride=1, padding=1),
              nn.ELU(True),

nn.Conv2d(in_dim, out_dim, kernel_size=1, stride=1, padding=0),
              nn.AvgPool2d(kernel_size=2, stride=2))
def deconv_block(in_dim, out_dim):
    return
nn.Sequential(nn.Conv2d(in_dim, out_dim, kernel_size=3, stride=1, padding=1),
              nn.ELU(True),

nn.Conv2d(out_dim, out_dim, kernel_size=3, stride=1, padding=1),
              nn.ELU(True),
              nn.UpsamplingNearest2d(scale_factor=2))

def blockUNet(in_c, out_c, name, transposed=False, bn=True,
relu=True, dropout=False):
    block = nn.Sequential()
    if relu:
        block.add_module('%s.relu' % name, nn.ReLU(inplace=True))
    else:
        block.add_module('%s.leakyrelu' % name, nn.LeakyReLU(0.2,
inplace=True))
    if not transposed:
        block.add_module('%s.conv' % name, nn.Conv2d(in_c, out_c, 4,
2, 1, bias=False))
    else:
        block.add_module('%s.tconv' % name, nn.ConvTranspose2d(in_c,
out_c, 4, 2, 1, bias=False))
```



```

    if bn:
        block.add_module('%s.bn' % name, nn.BatchNorm2d(out_c))
    if dropout:
        block.add_module('%s.dropout' % name, nn.Dropout2d(0.5,
inplace=True))
    return block

```

- 定义判别器

```

In [22]: class D(nn.Module):
    def __init__(self, nc, ndf, hidden_size):
        super(D, self).__init__()

        # 256
        self.conv1 =
nn.Sequential(nn.Conv2d(nc, ndf, kernel_size=3, stride=1, padding=1),
                nn.ELU(True))

        # 256
        self.conv2 = conv_block(ndf, ndf)
        # 128
        self.conv3 = conv_block(ndf, ndf*2)
        # 64
        self.conv4 = conv_block(ndf*2, ndf*3)
        # 32
        self.encode = nn.Conv2d(ndf*3, hidden_size,
kernel_size=1, stride=1, padding=0)
        self.decode = nn.Conv2d(hidden_size, ndf,
kernel_size=1, stride=1, padding=0)
        # 32
        self.deconv4 = deconv_block(ndf, ndf)
        # 64
        self.deconv3 = deconv_block(ndf, ndf)
        # 128
        self.deconv2 = deconv_block(ndf, ndf)
        # 256
        self.deconv1 =
nn.Sequential(nn.Conv2d(ndf, ndf, kernel_size=3, stride=1, padding=1),
                nn.ELU(True),

nn.Conv2d(ndf, ndf, kernel_size=3, stride=1, padding=1),
                nn.ELU(True),

```

```

nn.Conv2d(ndf, nc, kernel_size=3,
stride=1, padding=1),

nn.Tanh())

"""

self.deconv1 =
nn.Sequential(nn.Conv2d(ndf, nc, kernel_size=3, stride=1, padding=1),
nn.Tanh())

"""

def forward(self, x):
    out1 = self.conv1(x)
    out2 = self.conv2(out1)
    out3 = self.conv3(out2)
    out4 = self.conv4(out3)
    out5 = self.encode(out4)
    dout5= self.decode(out5)
    dout4= self.deconv4(dout5)
    dout3= self.deconv3(dout4)
    dout2= self.deconv2(dout3)
    dout1= self.deconv1(dout2)
    return dout1

```

- 定义生成器

```

In [23]: class G(nn.Module):
    def __init__(self, input_nc, output_nc, nf):
        super(G, self).__init__()

        # input is 256 x 256
        layer_idx = 1
        name = 'layer%d' % layer_idx
        layer1 = nn.Sequential()
        layer1.add_module(name, nn.Conv2d(input_nc, nf, 4, 2, 1,
bias=False))

        # input is 128 x 128
        layer_idx += 1
        name = 'layer%d' % layer_idx
        layer2 = blockUNet(nf, nf*2, name, transposed=False,
bn=True, relu=False, dropout=False)

        # input is 64 x 64
        layer_idx += 1
        name = 'layer%d' % layer_idx

```



```

        layer3 = blockUNet(nf*2, nf*4, name, transposed=False,
bn=True, relu=False, dropout=False)
        # input is 32
        layer_idx += 1
        name = 'layer%d' % layer_idx
        layer4 = blockUNet(nf*4, nf*8, name, transposed=False,
bn=True, relu=False, dropout=False)
        # input is 16
        layer_idx += 1
        name = 'layer%d' % layer_idx
        layer5 = blockUNet(nf*8, nf*8, name, transposed=False,
bn=True, relu=False, dropout=False)
        # input is 8
        layer_idx += 1
        name = 'layer%d' % layer_idx
        layer6 = blockUNet(nf*8, nf*8, name, transposed=False,
bn=True, relu=False, dropout=False)
        # input is 4
        layer_idx += 1
        name = 'layer%d' % layer_idx
        layer7 = blockUNet(nf*8, nf*8, name, transposed=False,
bn=True, relu=False, dropout=False)
        # input is 2 x 2
        layer_idx += 1
        name = 'layer%d' % layer_idx
        layer8 = blockUNet(nf*8, nf*8, name, transposed=False,
bn=False, relu=False, dropout=False)

    ## NOTE: decoder
    # input is 1
    name = 'dlayer%d' % layer_idx
    d_inc = nf*8
    dlayer8 = blockUNet(d_inc, nf*8, name, transposed=True,
bn=True, relu=True, dropout=True)

    #import pdb; pdb.set_trace()
    # input is 2
    layer_idx -= 1
    name = 'dlayer%d' % layer_idx
    d_inc = nf*8*2
    dlayer7 = blockUNet(d_inc, nf*8, name, transposed=True,

```

```

bn=True, relu=True, dropout=True)
    # input is 4
    layer_idx -= 1
    name = 'dlayer%d' % layer_idx
    d_inc = nf*8*2
    dlayer6 = blockUNet(d_inc, nf*8, name, transposed=True,
bn=True, relu=True, dropout=True)
    # input is 8
    layer_idx -= 1
    name = 'dlayer%d' % layer_idx
    d_inc = nf*8*2
    dlayer5 = blockUNet(d_inc, nf*8, name, transposed=True,
bn=True, relu=True, dropout=False)
    # input is 16
    layer_idx -= 1
    name = 'dlayer%d' % layer_idx
    d_inc = nf*8*2
    dlayer4 = blockUNet(d_inc, nf*4, name, transposed=True,
bn=True, relu=True, dropout=False)
    # input is 32
    layer_idx -= 1
    name = 'dlayer%d' % layer_idx
    d_inc = nf*4*2
    dlayer3 = blockUNet(d_inc, nf*2, name, transposed=True,
bn=True, relu=True, dropout=False)
    # input is 64
    layer_idx -= 1
    name = 'dlayer%d' % layer_idx
    d_inc = nf*2*2
    dlayer2 = blockUNet(d_inc, nf, name, transposed=True,
bn=True, relu=True, dropout=False)
    # input is 128
    layer_idx -= 1
    name = 'dlayer%d' % layer_idx
    dlayer1 = nn.Sequential()
    d_inc = nf*2
    dlayer1.add_module('%s.relu' % name, nn.ReLU(inplace=True))
    dlayer1.add_module('%s.tconv' % name,
nn.ConvTranspose2d(d_inc, output_nc, 4, 2, 1, bias=False))
    dlayer1.add_module('%s.tanh' % name, nn.Tanh())

```

```
self.layer1 = layer1
self.layer2 = layer2
self.layer3 = layer3
self.layer4 = layer4
self.layer5 = layer5
self.layer6 = layer6
self.layer7 = layer7
self.layer8 = layer8
self.dlayer8 = dlayer8
self.dlayer7 = dlayer7
self.dlayer6 = dlayer6
self.dlayer5 = dlayer5
self.dlayer4 = dlayer4
self.dlayer3 = dlayer3
self.dlayer2 = dlayer2
self.dlayer1 = dlayer1
```

```
def forward(self, x):
    out1 = self.layer1(x)
    out2 = self.layer2(out1)
    out3 = self.layer3(out2)
    out4 = self.layer4(out3)
    out5 = self.layer5(out4)
    out6 = self.layer6(out5)
    out7 = self.layer7(out6)
    out8 = self.layer8(out7)
    dout8 = self.dlayer8(out8)
    dout8_out7 = torch.cat([dout8, out7], 1)
    dout7 = self.dlayer7(dout8_out7)
    dout7_out6 = torch.cat([dout7, out6], 1)
    dout6 = self.dlayer6(dout7_out6)
    dout6_out5 = torch.cat([dout6, out5], 1)
    dout5 = self.dlayer5(dout6_out5)
    dout5_out4 = torch.cat([dout5, out4], 1)
    dout4 = self.dlayer4(dout5_out4)
    dout4_out3 = torch.cat([dout4, out3], 1)
    dout3 = self.dlayer3(dout4_out3)
    dout3_out2 = torch.cat([dout3, out2], 1)
    dout2 = self.dlayer2(dout3_out2)
    dout2_out1 = torch.cat([dout2, out1], 1)
```

```
dout1 = self.dlayer1(dout2_out1)
return dout1
```